Data Structure & Algorithm Lab

ECE2132



Department of Electronics & Communication Engineering

By:

Dr. D.V. Prashant
Dr. Kamal Kishore Upadhyay

Manipal University Jaipur,

Dehmi Kalan, Off Jaipur-Ajmer Expressway, Jaipur, (Raj.)

Rajasthan 303007

MANIPAL UNIVERSITY
JAIPUR
(University under Section 2(f) of the UGC Act)

(AY 2026-27)

Experiment No. 11

Write a Program to Implement Singly Linked List with Insertion, Deletion, and Traversal Operations.

AIM: To study and implement a Singly Linked List in C++ and perform insertion, deletion, and traversal operations.

SOFTWARE USED: Visual Studio Code / Code: Blocks / Dev-C++, GCC/G++ Compiler. Operating System: Windows/Linux

THEORY: A Linked List is a linear data structure in which elements are stored in separate units called nodes. Unlike arrays, linked list nodes are not required to be stored in consecutive memory locations.

A Singly Linked List consists of nodes, and each node contains two parts: a data field that stores the element and a pointer field that stores the address of the next node.

The first node is accessed using a pointer called head. The last node points to NULL, indicating the end of the list.

A singly linked list can grow or shrink dynamically as elements are inserted or deleted.

Singly Linked List

The singly linked list performs three main operations:

Insertion: The insertion operation is used to add a new node to the linked list.

Deletion: The deletion operation is used to remove an existing node from the linked list.

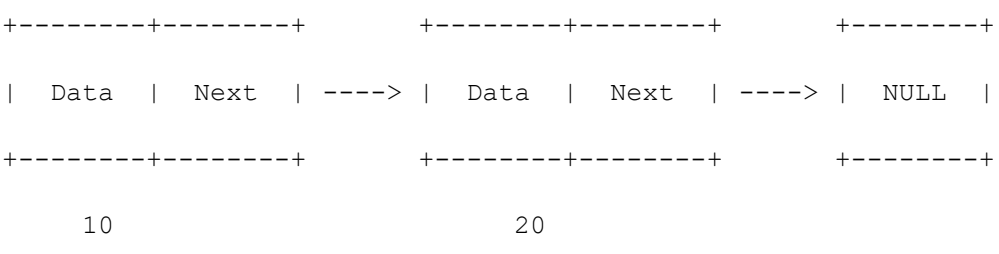
Traversal: The traversal operation is used to visit and display each node of the linked list, starting from the head node.

Structure of a Node

Each node contains two parts:

1. Data: Stores value.

2. Next: Stores the address of the next node.



Insertion Operation

Insertion adds a new node to the linked list. A node can be inserted at the beginning, at the end, or at a specified position.

Example: Insert 5 at the end.

Before insertion: 10 -> 20 -> 30 -> NULL

After insertion: 10 -> 20 -> 30 -> 5 -> NULL

Deletion Operation

Deletion removes a node from the linked list. The links are updated so that the remaining nodes stay connected.

Example: Delete 20.

Before deletion: 10 -> 20 -> 30 -> NULL

After deletion: 10 -> 30 -> NULL

Traversal Operation

Traversal visits each node from the head to the last node. The process continues until the pointer reaches NULL.

Example: 10 -> 20 -> 30 -> NULL

Output: 10 20 30

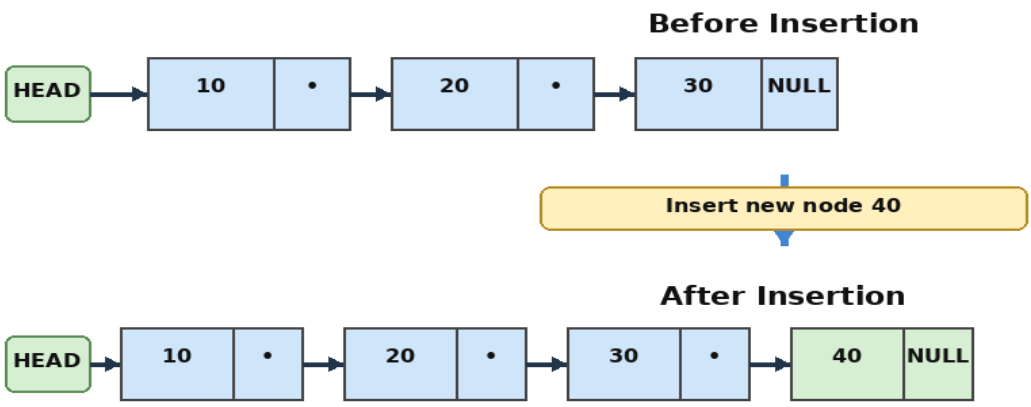


Figure.1: Insertion at the end of singly linked-list

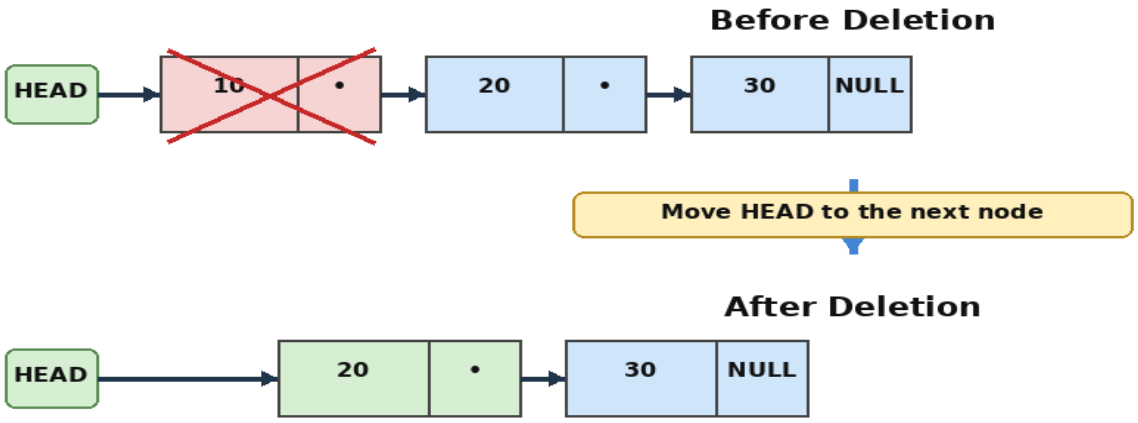


Figure.2: Deletion from the beginning.

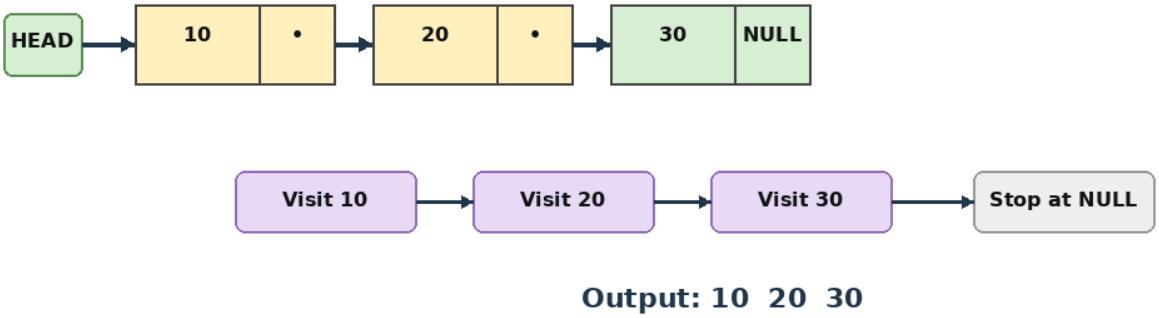


Figure.3: Traversal of a Singly Linked List

Feature	Insertion at End	Deletion at Beginning	Traversal
Purpose	Add a new node	Remove the first node	Visit all nodes
Pointer change	Last node's next points to the new node	HEAD moves to HEAD->next	Temporary pointer moves to the next node
Time complexity	O(n) in this program	O(1)	O(n)
Extra space per operation	O(1)	O(1)	O(1)
Can fail?	Allocation may fail	List may be empty	List may be empty

Figure. 5: Comparison of Singly linked list operations

ALOGRITHM:

Insertion Operation

1. Start.
2. Create a new node and store the given value.
3. If the list is empty, make the new node the head.
4. Otherwise, move to the last node.
5. Connect the last node to the new node.
6. Stop.

Deletion Operation

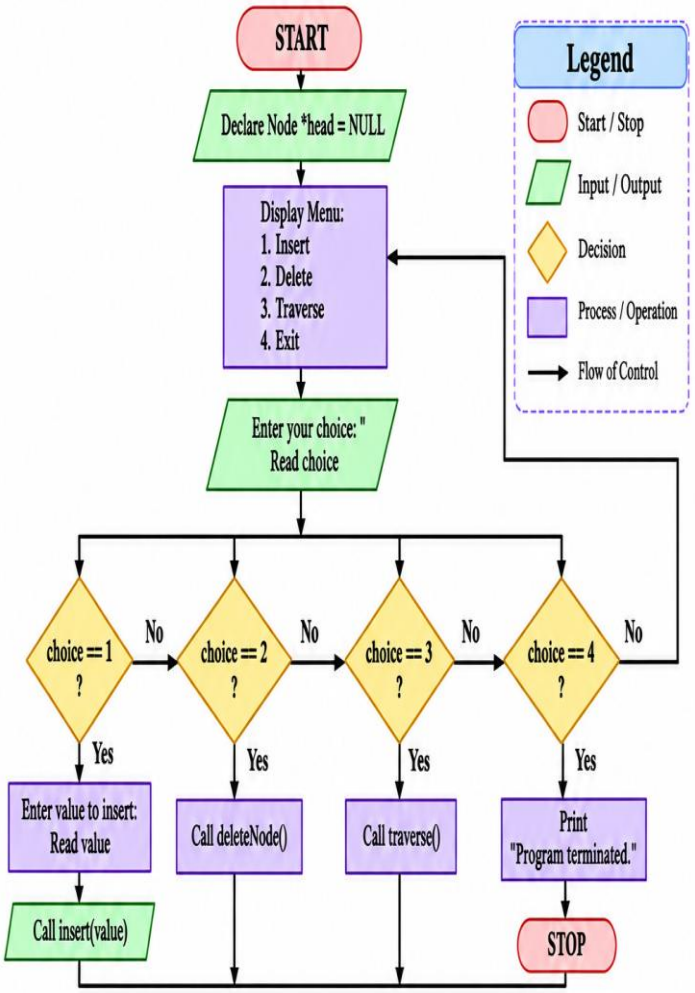
1. Start.
2. Check whether the list is empty.
3. If the list is empty, display a message.
4. Otherwise, store the head node temporarily.
5. Move the head pointer to the next node.
6. Delete the previous head node.
7. Stop.

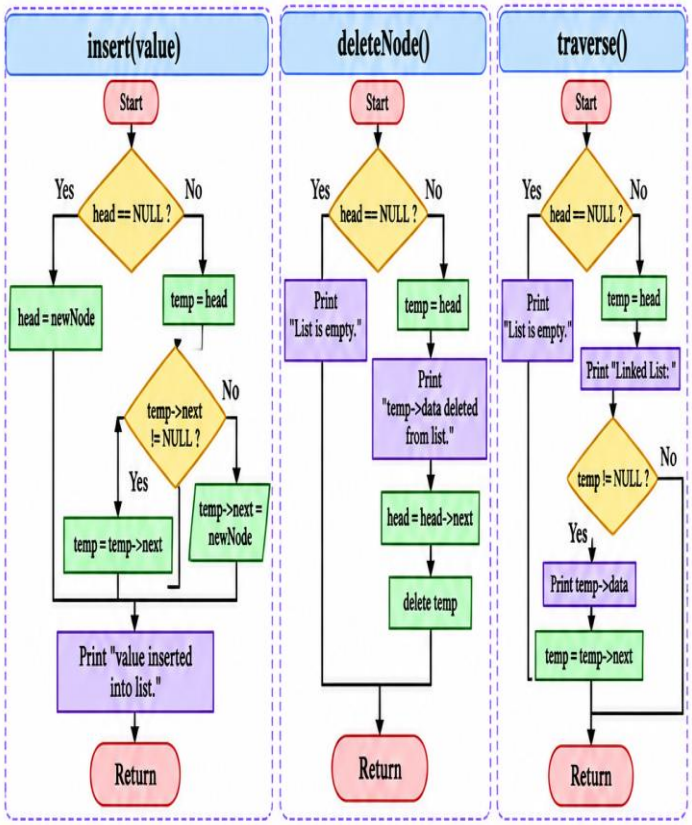
Traversal Operation

1. Start.
2. Check whether the list is empty.
3. Set a temporary pointer to the head.
4. Display the data of the current node.
5. Move the pointer to the next node.
6. Repeat until the pointer becomes NULL.
7. Stop.

PROGRAM (C++):

```
#include <iostream>
using namespace std;
struct Node
{
      int data;
      Node *next;
};
Node *head = NULL;
void insert(int value)
{
      Node *newNode = new Node;
      newNode->data = value;
      newNode->next = NULL;
      if (head == NULL)
      {
            head = newNode;
      }
      else
      {
            Node *temp = head;
            while (temp->next != NULL)
            {
                  temp = temp->next;
            }
            temp->next = newNode;
      }
      cout << value << " inserted into list." << endl;
}
void deleteNode()
{
      if (head == NULL)
      {
            cout << "List is empty." << endl;
      }
      else
      {
            Node *temp = head;
            cout << temp->data << " deleted from list.
            head = head->next;
            delete temp;
      }
}
void traverse()
{
      if (head == NULL)
      {
            cout << "List is empty." << endl;
      }
      else
      {
            Node *temp = head;
            cout << "Linked List: ";
            while (temp != NULL)
            {
                  cout << temp->data << " ";
                  temp = temp->next;
            }
            cout << endl;
      }
}
int main()
{
      int choice, value;
      do
      {
            cout << "\n1. Insert" << endl;
            cout << "2. Delete" << endl;
            cout << "3. Traverse" << endl;
            cout << "4. Exit" << endl;
            cout << "Enter your choice: ";
            cin >> choice;
            if (choice == 1)
            {
                  cout << "Enter value to insert: ";
                  cin >> value;
                  insert(value);
            }
            else if (choice == 2)
            {
                  deleteNode();
            }
            else if (choice == 3)
            {
                  traverse();
            }
            else if (choice == 4)
            {
                  cout << "Program terminated." << endl;
            }
            else
            {     cout << "Invalid choice!" << endl;
      } while (choice != 4);
      return 0;
}
```





EXPLANATION OF THE PROGRAM:

Statement / Function	Explanation
struct Node	Defines the structure of each node.
data	Stores the value of a node.
next	Stores the address of the next node.
head	Points to the first node.
insert()	Inserts a new node at the end.
deleteNode()	Deletes the first node.
traverse()	Displays all nodes in the list.
new Node	Dynamically creates a new node.
delete temp	Releases the memory occupied by the deleted node.
NULL	Indicates that there is no next node.

SAMPLE INPUT 1:

1. Insert

2. Delete

3. Traverse

4. Exit

Enter your choice: 1

Enter value to insert: 10

10 inserted into list.

Enter your choice: 1

Enter value to insert: 20

20 inserted into list.

Enter your choice: 1

Enter value to insert: 30

30 inserted into list.

Enter your choice: 3

CALCULATION:

Initially, the linked list is empty.

Insert 10:

HEAD -> 10 -> NULL

Insert 20:

HEAD -> 10 -> 20 -> NULL

Insert 30:

HEAD -> 10 -> 20 -> 30 -> NULL

Therefore, the linked list contains three nodes.

SAMPLE OUTPUT

Linked List: 10 20 30

SAMPLE INPUT 2

Delete the first node from the linked list.

Enter your choice: 2

CALCULATION

Before deletion:

HEAD -> 10 -> 20 -> 30 -> NULL

The first node, containing 10, is deleted and the head pointer moves to the next node.

After deletion:

HEAD -> 20 -> 30 -> NULL

SAMPLE OUTPUT

10 deleted from list.

After traversing the list again:

Linked List: 20 30

PERFORMANCE ANALYSIS

The performance of a singly linked list depends on the operation being performed.

Insertion Operation

In this program, insertion is performed at the end. The program traverses the list to find the last node.

Time Complexity = O(n)

Deletion Operation

Deletion is performed from the beginning. The head pointer is updated directly.

Time Complexity = O(1)

Traversal Operation

Traversal visits each node in the linked list.

Time Complexity = O(n)

TIME COMPLEXITY

Operation	Time Complexity
Insertion at end	O(n)
Deletion at beginning	O(1)
Traversal	O(n)

SPACE COMPLEXITY

A separate node is created for every element inserted into the linked list. Each node stores data and a pointer to the next node.

Space Complexity = O(n)

CHARACTERISTICS OF SINGLY LINKED LIST

1. It is a linear data structure.
2. Each node contains data and a pointer to the next node.
3. Nodes do not need to occupy consecutive memory locations.
4. The first node is accessed using the head pointer.
5. The last node points to NULL.
6. Insertion and deletion can be performed by changing pointers.
7. Memory is allocated dynamically.
8. Traversal is possible only in the forward direction.
9. The size can grow or shrink during program execution.
10. Each node requires extra memory for its pointer.

IMPLEMENTATION OF CODE

```cpp
#include <iostream>
using namespace std;
struct Node
{
    int data;
    Node* next;
};
Node* head = NULL;

void insert(int value)
{
    Node* newNode = new Node;
    newNode->data = value;
    newNode->next = NULL;

    if(head == NULL)
    {
        head = newNode;
    }
    else
    {   Node* temp = head;

        while(temp->next != NULL)
        {
            temp = temp->next;
        }
    }
    cout << value << " inserted into list." << endl;
}

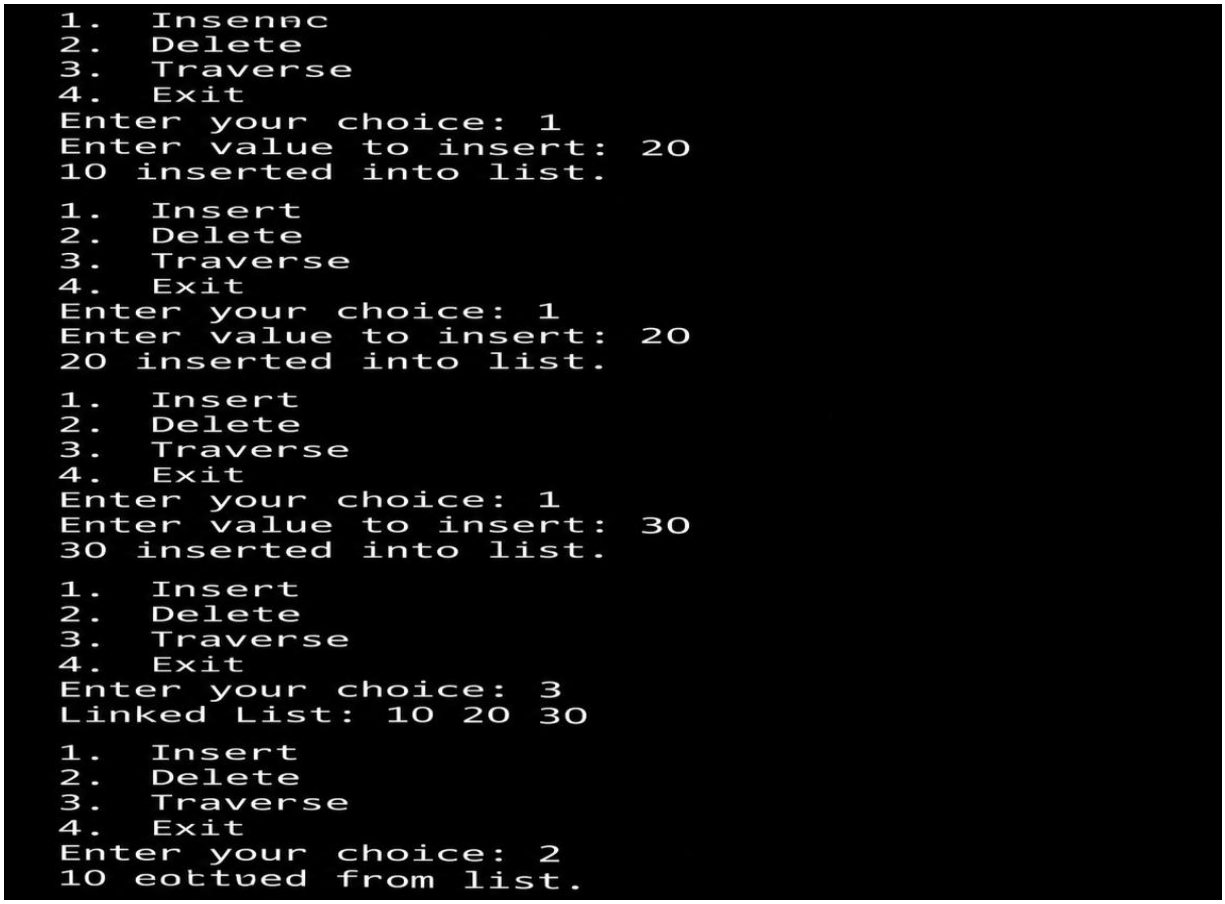
void deleteNode()
{
    if (head == NULL)
    {
        cout << "List is empty." << endl;
    }
    else
    {   Node* temp = head;
        cout << temp->data << " deleted from list." << endl;
        head = head->next;
    }   delete temp;
}

void traverse()
{
    if (head == NULL)
    {
        cout << "List is empty." << endl;
    }
    else
    {   Node* temp = head;
        cout << "Linked List: ";
        while(temp != NULL)
        {
            cout << temp->data << " ";
        }
        cout << endl;
    }
}
int main()
{
    int choice, value;
    do
    {   cout "1. Insert;
        cout "\2. Delete",
        cout "\n3. Traverse";
        cout "\n4. Exit",
        "Enter your choice: ";
        cin >> choice;
        if (choice == 1)
        {
            cout "Enter value to insert: ";
            cin  >> value;
        }    insert(value);
        else if (choice == 2)
        {    deleteNode();
        else if (choice == 3)
        {    cout << "ermioen" << endl;
        else
    } while (choice != 4);
}   return 0;

10. choice.e"
21. Delete< "embdee
1; Exit;

void insema(iULL.4)
{
    if (head == head;
    {
        head << newNode;
    }
    else
    {   Node* temp = 1)

        while (temp->next != NULL)
        {
            temp = temp->next;
        }
        temp->next = newNode;
    }
    cout << value << " inserted into list." << endl;
}
```



APPLICATIONS: Singly linked lists are used in implementing stacks and queues, dynamic memory management, maintaining lists of records, representing graph adjacency lists, implementing playlists and sequential collections, and situations where frequent insertion and deletion are required.

RESULT:

The program was successfully executed. It demonstrated the implementation of a Singly Linked List in C++ and performed insertion at the end, deletion from the beginning, and traversal operations successfully.

ASSIGNMENT:

- WAP to insert a node at the beginning of a singly linked list.
- WAP to insert a node at the end of a singly linked list.
- WAP to delete a node from the beginning of a singly linked list.
- WAP to delete a node from the end of a singly linked list.
- WAP to search for an element in a singly linked list.
- WAP to count the total number of nodes.
- WAP to display the linked list in reverse order.
- WAP to insert a node at a specified position.
- WAP to delete a node at a specified position.